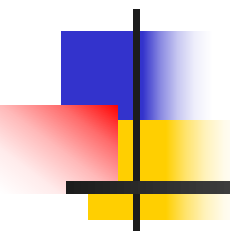
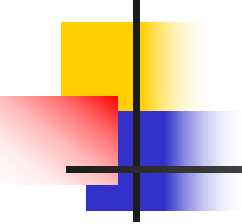


多维查询处理及优化



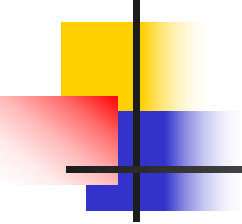
MDX多维查询语言

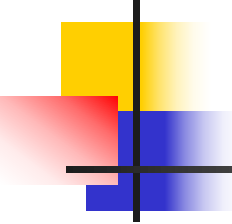


OLE DB for OLAP

OLE DB for OLAP是一种标准，它由微软发布，提供了对多维数据存取的接口和方法。

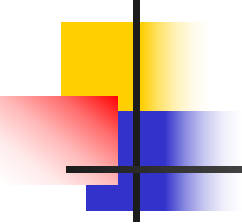
它的特点是独立于多维数据的具体存储结构。其中用于访问多维数据源的多维表达式 (Multi-Dimension eXpression--MDX)，提供了语义丰富的查询多维数据的语句，自发布以来逐渐成为工业界的事实标准。

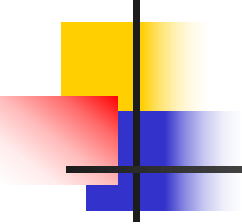
- 
- MDX类似于关系数据库中的标准SQL语句。创建MDX的主要目的是为了更方便用户对多维数据进行访问。
 - MDX是一种表达式语言，它的函数可以用来定义新的计算成员，创建局部的数据方体，或者通过OLE DB或者ADO对数据方体中的数据进行访问。
 - 在MDX中，用户还可以自己定义函数，来满足某些特殊要求。

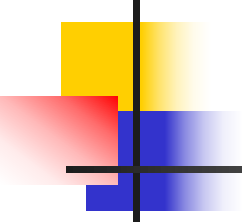


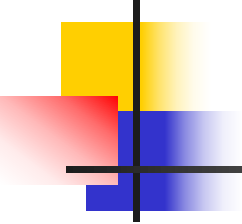
MDX与SQL

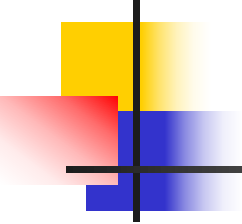
- ❖ 在关系数据库中，用户查询的对象是关系，其结果集也是一个关系，对多维数据，查询的对象是方体，查询的结果集也是方体。
- ❖ 对于关系数据库的查询我们采用标准的**SQL**语句，对于多维数据的访问，可以采用专用于访问多维数据的**MDX**，当然也可以采用扩展的**SQL**语句。

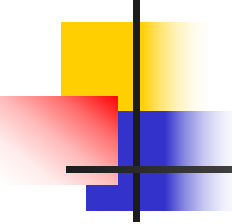
- 
-
- 由于大家对SQL都比较熟悉，所以MDX的语法在很多方面力求和SQL保持一致。
 - 很多用MDX实现的功能用SQL也能实现，尽管实现的手段各异。
 - 不过，MDX查询语言使得很多功能的实现更加有效、更为直观，尤其MDX更适合多维数据的操作

- 
-
- MDX语言和SQL语言一样，语句具有SELECT、FROM和WHERE结构。
 - 但MDX语言采用其他的关键字来查询数据方体，返回可以进一步加以分析的数据，并提供了可以对返回的数据进行操作的相应的函数。
 - 而且，MDX语言增强了用户自定义函数的功能，可以扩展或者定制用户某些特殊的商业需求。

- 
- MDX语言所提供的数据定义语言(DDL)也和SQL类似。
 - 例如, 提供了对数据方体、维、度量等OLAP环境下的对象进行创建(creating)、修改(modifying)、删除(deleting)的命令

- 
-
- SQL的数据定义语言/操纵语言都是返回或操纵两维的数据，而MDX用来返回或操纵多维的数据
 - 为了定义和抽取数据，不管是针对单个数据单元（**cell**）还是多个数据单元（统称为一个数据集），MDX都采用一套坐标系统来对它们进行定义

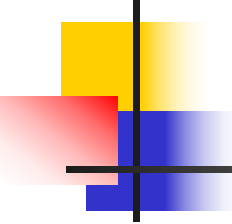
- 
-
- 该坐标系中的每一个坐标轴上可以有多个维。各个维上成员的不同组合就构成了该坐标轴上的不同的坐标点。



多维模式

多维数据最基本的结构模式是方体 (**cube**)，每个方体由多个维组成，每个维上可以有多个层次，并且每个维上有多个成员。例如，一个关于产品销售数据的数据方体由以下**6**个维组成：

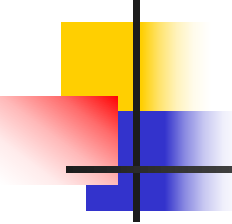
- SalesRep
- Geography，它其中的一个层次 (**hierarchy**) 有以下层 (**level**)：Continents, Countries 和 States.
- Quarters
- Years
- Measures，包括 Sales, PercentChange 和 BudgetedSales
- Products



多维数据集合

在多维数据存储中，数据是以 n 维方体中的数值点存在的，用户可以指定一个查询来访问多维数据，用户查询对象可以是一个或多个方体，其结果集是一个 p 维的方体（ $p \leq n$ ），我们称之为数据集合。

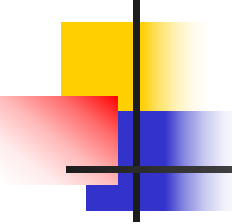
数据集合封装了多维查询的结果集，它是执行一个MDX语句的结果集。数据集合是由一组轴（axis）组成，轴上的点称为坐标点（coordinate）。每个轴上可以包含多个维。



For Example

例如，要对一个具有4个维的数据方体查询：**Geography, Products, Measures**和 **Quarters**，其中有两个度量值：**Sales** 和**Cost**。现在用户需要了解**USA**在每个季度每种产品的销售情况，则应该产生如下的结果数据集合：

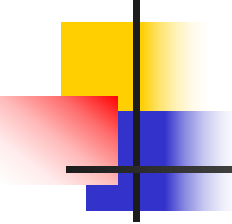
	Sales for USA		
	Computers	ExpansionCards	Floppy Drivers
Qtr1	4622	1245	2356
Qtr2	34069	25	235847
Qtr3	39	429058	5920
Qtr4	53984	432	42908



MDX的基本概念

MDX语句必须包含下面的信息：

1. 查询所涉及的数据方体
2. 产生的结果数据集合的轴的个数
3. 数据方体的维与结果数据集合中轴的对应关系
4. 在每个维上的成员



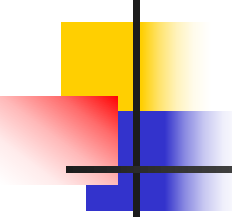
MDX的基本概念(续)

一个典型的MDX语句具有下面的格式:

```
SELECT <axis_specification> [, <axis_specification>...]  
FROM <cube_specification>  
WHERE < slicer_specification>
```

在编写MDX语句时, 通常把所查询的数据方体的维集合划分为两个子集合:

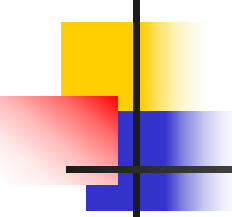
- ◆ 轴维 (Axis dimensions), 在该维上通常取多个成员;
- ◆ 切片维 (Slicer dimensions), 在该维上通常取一个成员;



MDX的基本概念(续)

轴维: 一个轴上可以对应多个维，每个轴是由一组元组的集合构成的，如果轴只对应一个维，则其中的每个元组就是一个成员，如果轴对应n个维，则每个元组由n个来自不同维的成员组成。

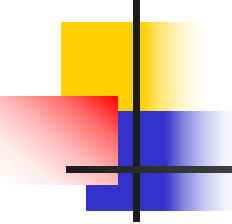
	Jack		Tom	
	Computers	Floppy Drivers	Computers	Floppy Drivers
Qtr1	4622	2356	4622	2356
Qtr2	34069	235847	34069	235847
Qtr3	39	5920	39	5920
Qtr4	53984	42908	53984	42908



MDX的基本概念(续)

切片维: 在该维上通常取一个成员; 相当于SQL中的where条件,如用户需要了解USA在每个季度每种产品的销售情况, 则应该产生如下的结果数据集合,在该例中, Quarter和Product维是轴维, Measure和Geography是切片维。

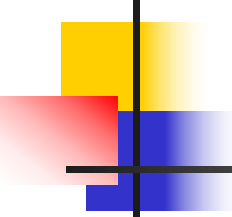
	Sales for USA		
	Computers	ExpansionCards	Floppy Drivers
Qtr1	4622	1245	2356
Qtr2	34069	25	235847
Qtr3	39	429058	5920
Qtr4	53984	432	42908



MDX的基本概念(续)

指定一个轴的内容可以按照以下的步骤:

1. 创建一个表达式来生成成员;
2. 获得在每个轴的每个维的成员;
3. 对这些成员做笛卡氏集, 生成每个轴的元组集合。

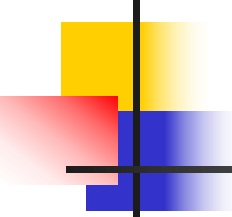


MDX的基本概念(续)

选择一组成员有多种方法，例如：

- ◆ 枚举法，例如：{Venkatrao, Netz}
- ◆ 集合表达式 (set expression)

例如：表达式{USA.CHILDREN}生成下面的集合：{USA_North, USA_South}, 表达式{USA_North.CHILDREN, USA_South, Japan}生成集合：{Seattle, Boston, USA_South, Japan}



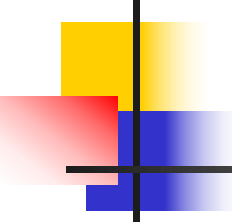
MDX的基本概念(续)

在一个数据集合中可以有多轴，用户为每个轴指定了元组集合后，还需要把这些元组集合投影到指定的轴上。这在MDX语句中<axis_specification>部分进行说明：

`<axis_specification> ::= <set> ON <axis_name>`

`<axis_name> ::= COLUMNS | ROWS | PAGES | SECTIONS | CHAPTERS | AXIS(index)`

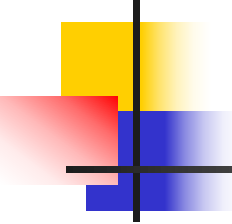
前5个轴可以用COLUMNS, ROWS, PAGES, SECTIONS和CHAPTERS表示。OLE DB for OLAP要求至少支持3个或以上个轴。



MDX的基本概念(续)

指定切片维的值: 查询的数据方体的维中, 没有
在轴上的维都认为是切片维。在MDX语句
中, 可以用下面的语句来表示切片条件:

WHERE (Sales, [2020], Products.[All])



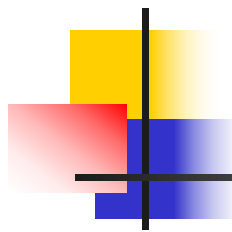
MDX的基本概念(续)

例如:一个多维数据库**Sales**由季度、年、地区、产品、销售代表和销售额六个维组成。其中销售额是度量维。时间维上的层次为 *日,月,季,年*, 地区维上的层次为 *州,地区,国家*。

现在: 要查询销售代表**Venkatrao**和**Netz**在2020年的**1月、2月、3月、第二季度、第三季度、10月、11月、12月**分别在美国南部的各个州、美国北部及日本的销售业绩 (用销售额表示) 。

MDX的基本概念(续)

		Venkatrao				Netz					
		USA				Japan	USA			Japan	
		USA_NE		USA_SE			USA_NE				USA_SE
		Seattle	Boston				Seattle	Boston			
Qtr1	Jan	00	10	20	30	40	50	60	70		
	Feb	01	11	21	31	41	51	61	71		
	Mar	02	12	22	32	42	52	62	72		
Qtr2		03	13	23	33	43	53	63	73		
Qtr3		04	14	24	34	44	54	64	74		
Qtr4	Oct	05	15	25	35	45	55	65	75		
	Nov	06	16	26	36	46	56	66	76		
	Dec	07	17	27	37	47	57	67	77		



MDX的基本概念(续)

该数据集合具有以下的特点:

1.轴维 (Axis dimensions) : Quarters, SalesRep, Geography

■ 2.切片维 (Slicer dimensions) : Measures, Years, Products

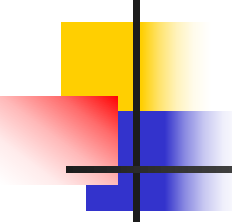
■ 3. 具有两个轴: COLUMN, x-axis, or axis 0;

ROW, y-axis, or axis 1

■ 4. x-axis: SalesRep 和 Geography dimensions (后者嵌套在前者之中)

■ 5. y-axis: Quarters dimension

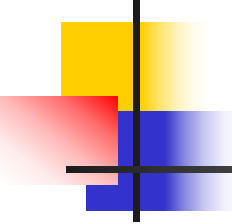
■ 6. 切片条件: WHERE (Sales, [2020], Products.[All])



MDX的基本概念(续)

生成数据集合的MDX语句表示如下:

```
SELECT CROSSJOIN({Venkatrao, Netz},  
  {USA_North.CHILDREN, USA_South, Japan}) ON COLUMNS,  
  {Qtr1.CHILDREN, Qtr2, Qtr3, Qtr4.CHILDREN} ON ROWS  
FROM SalesCube  
WHERE (Sales, [2020], Products.[All])
```



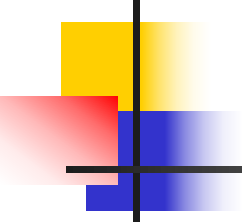
MDX语法

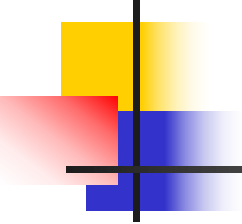
有关MDX语法格式：

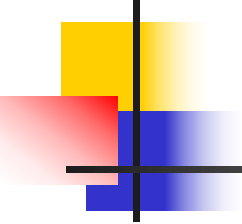
请参照微软MDX语言相关文档：

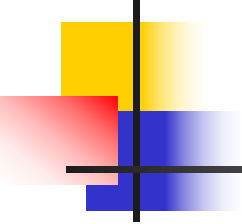
<http://www.microsoft.com/data/oledb/olap/prodinfo.html>]

ParaWare实现时对其进行裁减，实现其基本功能。

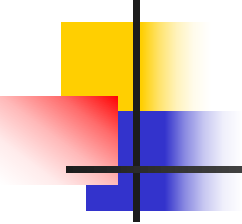
- 
-
- 终端用户的多维查询请求可以直接表达成**MDX**语句的形式，也可以由**OLAP**系统的客户端工具将用户的钻取、旋转等多维操作转换成相应的**MDX**查询语句。

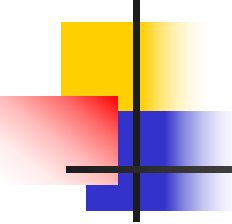
- 
-
- 对MDX查询语句的处理，有两种方式。
 - 一种是将其转变成SQL,然后采用RDBMS的查询处理引擎来实现。
 - 另一种是直接对MDX语句进行处理，对其作语法检查、语义分析、查询处理和优化、执行并最终得到查询结果。

- 
- 第一种方法的好处是可以充分利用现有SQL的查询处理和优化技术，但由于SQL并不是专为多维数据查询设计的，存在一些固有的限制。
 - 因此，人们更看好第二种方法，但到目前为止，尚未有详细的资料公开发表。
 - 在这里,介绍自行研制开发的并行数据仓库系统PARAWARE (Parallel Data Warehouse) 中所采用的方法

- 
-
- 由于一个MDX查询常常涉及一个维的多个层，例如
 - 在时间维上同时涉及月和季两层，
 - 在地区维上同时涉及城市和国家两层

而数据是以不同维、层的组合所构成的cuboid为单位存储的。

- 
-
- 因此，在处理MDX查询的时候，首先需要将一个MDX查询按照查询所涉及的维层的不同划分成多个子查询，每个子查询对应一个cuboid。



并行数据仓库Paraware简介

- 支持超大数据量存储
- 支持**MDX**多维查询语言
- 具有高查询处理性能

PareWare的体系结构

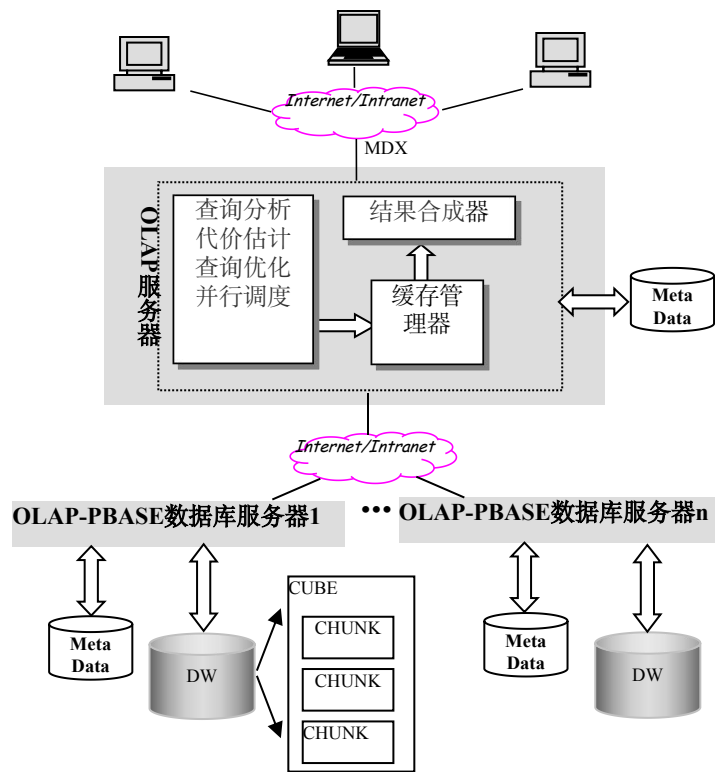
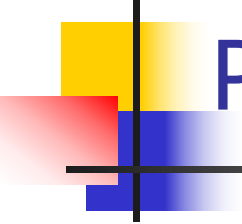


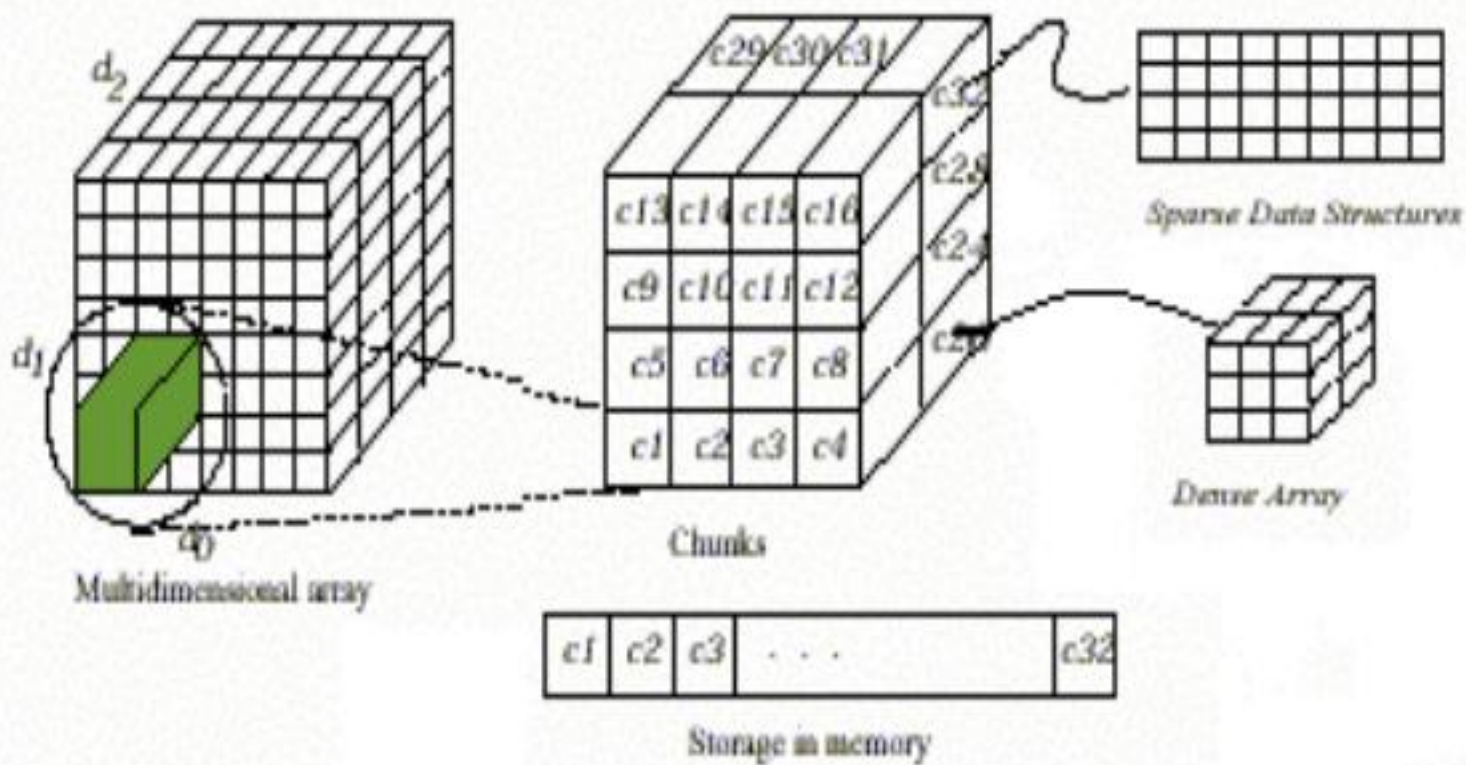
图1。ParaWare的体系结构



Paraware的多维存储

- **混合存储**: 将MOLAP和ROLAP两种存储方式相结合, 充分利用二者的优点, 克服其缺点。

混合存储





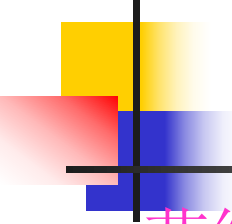
特殊关系表及其索引机制

■ 特殊关系表与传统关系表相比

- 相同处：通过索引按数据内容检索数据
- 不同处：特殊关系表只存储度量值，不存维值

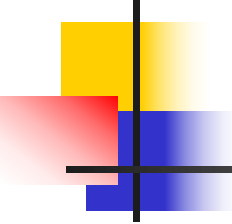
■ 两极索引机制

- 对数据块的索引：在划分维上建立
- 对数据块内部数据的索引：在非划分维上建立



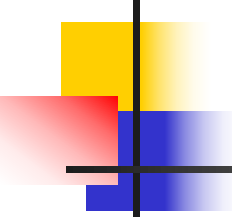
混合存储的优点

- **节约存储空间**，根据数据稀疏程度的不同，不同的数据块可以采用不同的数据存储方式，提高了存储效率；
- **提高计算速度**，经过划分以后数据分布在多个处理机上可以实现并行计算，而且可以设计高效的内存算法，提高了计算速度；
- **减少了数据访问时读取磁盘的平均次数。**



Paraware查询处理和优化

- 构造MDX语句
- 词法、语法分析及语义检查
- 子查询的划分
- 子查询的执行
- 组装查询结果



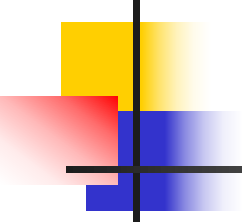
子查询的划分

■ 划分原因

- 不同的用户所关心的维空间不同
- 便于进行并行存取数据，提高查询效率

■ 划分依据

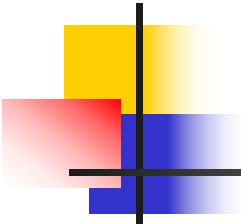
- 不同维层的组合

- 
-
- 一个MDX查询可以划分成多个子查询。每个子查询也是由多个轴、每个轴由多个维、每个维由多个成员构成，不同的是：在子查询中，每个维上所取的成员必须属于同一个维层。



MDX查询举例

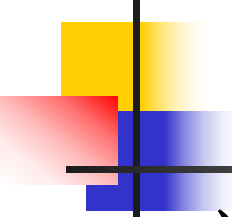
Sales in 1991			销1	销1	销1	销2	销3	销3	销4
			北京		上海	北京	北京	上海	北京 朝阳
			海淀	朝阳			海淀		
1月	电器								
	电器	洗衣机							
		电视							
		电冰箱							
2月	电器								
	电器	洗衣机							
		电视							
		电冰箱							
3月	电器	电视							
一季度	电器								



			销1	销2	销3	销1		销3	销4
			上海	北京	上海	北京		北京	北京
						海淀	朝阳	海淀	朝阳
一季度	电器								
1月	电器								
2月	电器								
1月	电器	洗衣机							
		电视							
		电冰箱							
2月	电器	洗衣机							
		电视							
		电冰箱							
3月	电器	电视							

子查询的执行

- 首先到缓存中去查数据, 找到, 则取出数据直接返回;
- 缓存中找不到, 则查数据字典, 看是否存在已经实体化的视图可以用来回答该查询, 若有, 则向存储层发送该查询及相应的视图号, 存储层直接取出数据返回结果。
- 如果没有已经实体化好的视图, 则需要选择一个计算代价最小的实体化视图, 对该查询进行实时聚集计算。



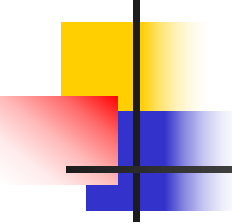
选择代价最小的实视图

- 视图可用性条件:
 - ❖ 视图的选择区域和查询的选择区域部分重叠
 - ❖ 存在导出关系
- 计算代价:

$$t_{scan}(v) = \frac{size(v)}{pagesize} \times t_{I/O}$$

$$t_{aggregate}(q, v) = t_{map}(v) + size(v) \times t_{cpu}(q, v)$$

$$cost(q, v) = t_{scan}(v) + t_{aggregate}(q, v)$$

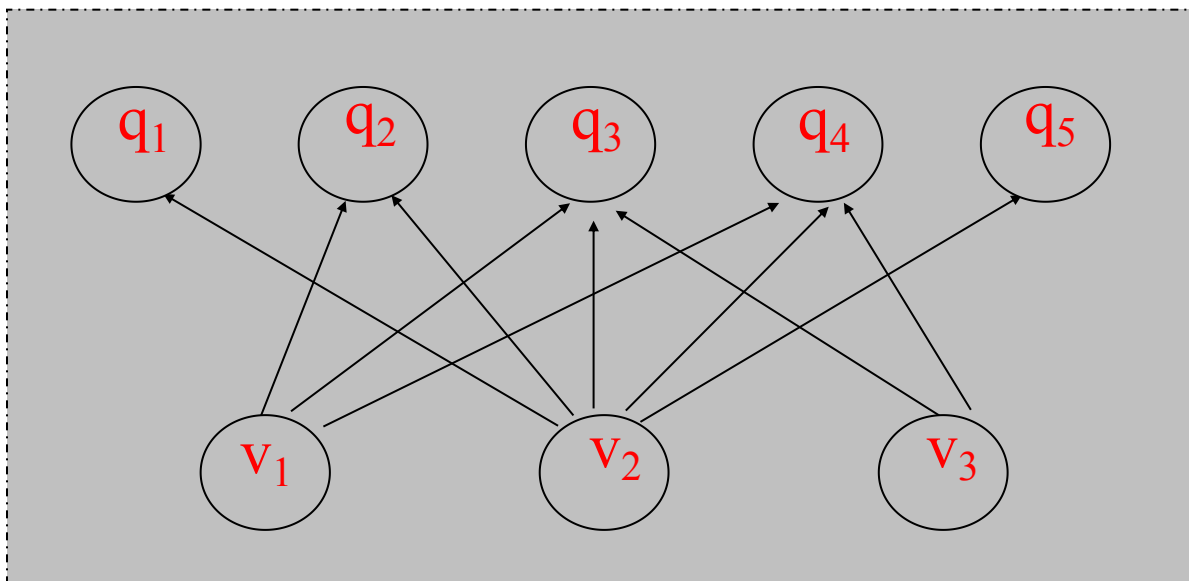


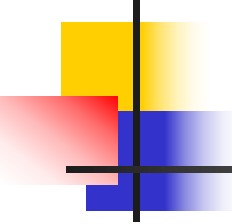
多查询优化策略

- 基于实体化视图的多查询优化
- 公共子查询的识别、提取和合并

基于实体化视图的多查询优化

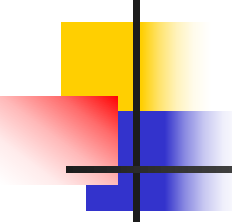
查询-视图图 $G=(Q \cup V, E)$





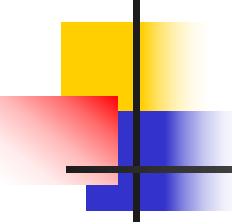
启发式算法

- 基本思想：按照查询-视图图中每个查询结点的入度由小到大将查询逐个加入到全局查询执行计划中。
(现有的算法是按照查询的聚集粒度由细到粗进行的)。



一劳永逸算法

- 启发式算法需要循环 k 次，最坏情况下（当每个查询被所有的视图覆盖时）， k 等于视图的数目。
- 一劳永逸算法：将多查询优化问题转换成单源最短路径问题，然后使用Dijkstra算法来为多个查询同时选定最好的实体化视图集合。



提取公共子查询

- 公共子查询的提取策略：弱共享方法
 - 找到查询子空间相同的子查询，将这些子查询提取出来予以合并即可得到公共子查询。
- 能够提取公共子查询的情况
 - 维成员全部相同的查询
 - 维相同，成员部分相同，部分不同的查询
 - 维相同，成员完全不同的查询